Capability Monotonicity in a Formal Skeleton for Agentic AI Safety

Manish Bhatt

Contents

Contents i

1 Abstract 1

2 How to read this thesis 3

3 Introduction 5
3.1 What survives the rule 5
3.2 What does not survive 5
3.3 Thesis statement, scaled to what remains 6
3.4 Structure 6

4 Related Work 7
4.1 Hybrid state and partial observability 7
4.2 Specification automata 7
4.3 Capability and reachability 8
4.4 What this chapter does not place 8

5 The Formal Model 9
5.1 The augmented state 9
5.2 Belief as an interface 9
5.3 Intervention and composition 10
5.4 Capability ordering 10

6 The Specification Automaton 13
6.1 The automaton 13

7 The Shield, and Capability Monotonicity 15
7.1 The shield interface 15
7.2 Capability monotonicity 16

8 A Recoverability-Monitor Interface 19

9 What Was Cut, and Why 21
9.1 The robust failure value and the viability kernel 21

9.2 Tail severity and adversarial energy 21
9.3 Recoverability as a probability, and the hysteresis prediction . . . 22
9.4 The runtime-assurance fallback rule 22
9.5 Causal-incentive analysis and the Defense Trilemma reading . . . 22
9.6 The eight-item empirical identification program 22
9.7 What this leaves 23

10 Conclusion 25

A thesis submitted in partial fulfilment of the requirements for the degree of Doctor of Philosophy.

1 Abstract

An earlier draft of this project proposed a full dynamic-programming theory of agentic AI safety — a robust reachability value, a viability kernel, a critical surface, a recoverability boundary, an adversarial-energy metric, a memory-hysteresis prediction, and an eight-item empirical program built on top of them. This document is not that draft. It is a rewrite under a rule borrowed from its companion thesis, **Two Schemas of Impossibility**: a claim appears only if it is proved in an elaborated Lean block or resolves to a named Lean declaration: it is a thesis about what remains of a larger proposal once every claim without a Lean declaration behind it is removed.

What remains is a small formal skeleton — an augmented agent state, a belief interface, an intervention/composition mechanism, a capability partial order, a specification automaton, a shield interface, and a recoverability-monitor interface, each a Lean `structure`, `inductive`, or `def` checked by the compiler as this document builds — together with one genuine theorem, `capability_reachable_subset`: a capability ordering that only ever adds executable actions can only ever enlarge the reachable state graph. The theorem is a general fact about monotone reachability graphs; its relevance to agentic AI safety is a reading argued for in the text, not a further mathematical result.

Chapter 7 lists every claim that was cut to reach this document — the value function, the viability kernel, the critical surface, the tail- severity and adversarial-energy functionals, the recoverability probability and its hysteresis prediction, a runtime-assurance design rule, a causal-incentive reading, and the empirical identification program — and says why each was cut: none of them corresponded to a Lean declaration. This thesis's claim about itself is only that the skeleton and the one theorem in Chapters 3 through 6 are genuinely backed, and that everything listed in Chapter 7 is a research proposal, not a result.

2 How to read this thesis

This is a short document by design. Chapter 1 states the scope reduction directly. Chapter 2 places the surviving formal objects, briefly, against their literatures. Chapters 3 through 6 give the Lean content: the state and capability structures, the specification automaton, the shield interface and the capability-monotonicity theorem, and the recoverability-monitor interface. Chapter 7 is an explicit accounting of what was removed and why. Chapter 8 concludes.

3 Introduction

This thesis started from a much larger proposal: a full dynamic-programming theory of agentic AI safety, with a robust reachability value, a viability kernel, a critical surface, a recoverability boundary, an adversarial-energy metric, a hysteresis prediction, and an eight-point empirical program. That proposal is not what follows. It is not what follows because none of it was backed by a machine-checked declaration, and this thesis's governing rule, inherited from its companion volume **Two Schemas of Impossibility**, is that a claim without Lean backing does not appear in the document. The companion thesis enforces this with a build-time citation gate: every numbered claim must either be proved in an elaborated code block or resolve to a named declaration in one of its four companion libraries, on pain of the build failing. This thesis adopts the same rule and applies it to itself, and the result is a much smaller document than the one first proposed.

3.1 What survives the rule

Four objects survive. Chapter 3 gives the augmented agent state, a belief interface, the intervention/composition mechanism, and the capability partial order, each as a Lean `structure`, `inductive`, or `def` that the compiler checks for well-formedness as this document builds. Chapter 4 gives the specification automaton that a trajectory-level safety property compiles into. Chapter 5 gives a shield interface and proves, as a genuine machine-checked theorem, that a capability ordering which only ever adds executable actions can only ever enlarge what is reachable — not a deep result, but a real one, `capability_reachable_subset`, checked by the Lean kernel rather than asserted in prose. Chapter 6 gives a recoverability monitor interface, offered explicitly as bookkeeping rather than as backing for any claim about what recoverability **is**.

3.2 What does not survive

Chapter 7 lists what was cut and why, so that a reader who saw an earlier, larger version of this project — or who read the informal writeup this thesis grew out of — can see exactly where the boundary was drawn and can check that the cut

was not selective. The short version: a robust failure value, a viability kernel, a critical surface, a tail-severity functional, an adversarial-energy metric, a hysteresis prediction, a runtime-assurance fallback rule, a reading of causal-incentive analysis, and an eight-item empirical program were all proposed at one point and all removed, because none of them was a Lean declaration and this thesis's rule does not make an exception for an idea being interesting.

3.3 Thesis statement, scaled to what remains

A small fragment of agentic-AI-safety reasoning — the state an agent occupies, the actions available to it under a capability ordering, the automaton a trajectory-level specification compiles into, and the fact that enlarging capability can only enlarge what is reachable — can be given a machine-checked formal encoding in Lean 4, and exactly one nontrivial consequence of that encoding, capability monotonicity of reachability, is proved rather than asserted.

This is a narrower claim than the one the larger proposal made, and it is narrower on purpose. A reader who wants the larger theory — the dynamic game, the critical surface, the empirical program — should read it as a research proposal, stated as such, and should not mistake anything in Chapters 3 through 6 of this document for evidence toward it.

3.4 Structure

Chapter 2 places the surviving formal objects against their literature. Chapter 3 gives the state, belief, intervention, and capability structures. Chapter 4 gives the specification automaton. Chapter 5 gives the shield interface and the capability-monotonicity theorem. Chapter 6 gives the recoverability-monitor interface. Chapter 7 lists what was cut, and why. Chapter 8 concludes.

4 Related Work

The formal objects that survive into Chapters 3 through 6 are small instances of larger, established constructions. This chapter places each one, and is explicit that placing an object against a literature is not the same as claiming this thesis reproduces that literature's theorems: the Lean content of this thesis is the four structures and the one theorem listed in Chapter 1, and nothing said here should be read as extending that content.

4.1 Hybrid state and partial observability

Chapter 3's augmented state, with continuous and discrete components living side by side, is an instance of the standard object in stochastic hybrid systems theory, where reachability results are already established for systems combining both. Chapter 3's belief interface is the standard sufficient-statistic construction from partially observable planning: the posterior over the hidden state given the observation history is what a planner needs to act optimally under partial observability, and belief-space planning is the classical way to reduce a partially observed problem to a fully observed one over that posterior. This thesis's `Belief` structure captures only the interface — a point estimate and an update rule — not the measure-theoretic content, deliberately: nothing downstream in this thesis needs more than the interface.

4.2 Specification automata

Compiling a trajectory-level temporal property into a monitor automaton, which Chapter 4's `SpecAutomaton` does, is the standard construction from runtime verification and from shielding in safe reinforcement learning: a temporal-logic specification is compiled into an automaton whose state tracks progress toward, or violation of, the property, so that a trajectory-level question becomes an ordinary question about the automaton's current state. Shielding then overrides a learner's action only when doing otherwise would violate the specification, which is the idea Chapter 5's `Shield` structure encodes, again as an interface rather than as a proof that any concrete shield built on it satisfies the specification.

4.3 Capability and reachability

The claim proved in Chapter 5 — that a capability ordering which only adds executable actions can only enlarge the reachable set — is an instance of an entirely general fact about reachability in monotone systems, unrelated to agents specifically: it holds for any state-transition graph where a ”stronger” edge set is a superset of a ”weaker” one. Its relevance to agentic AI safety is the reading, not the mathematics: shell access, unrestricted networking, credentials, write access, and delegation are each naturally modeled as enlarging the executable-action set at every state that has them, and the theorem then says the resulting reachable set can only grow. This reading is argued for in Chapter 5; the theorem itself needs nothing about agents to be true.

4.4 What this chapter does not place

Belief-space robustness under model uncertainty, temporal reachability values, runtime-assurance switching, information-flow reference monitors, and causal-incentive analysis all have substantial literatures that an earlier draft of this thesis related its (since-removed) claims to. Chapter 7 records that those claims were removed and does not restore this chapter’s discussion of their literatures, because a related-work section for a claim that no longer appears in the thesis would misrepresent what the thesis contains.

5 The Formal Model

This chapter fixes the objects that are actually declared in Lean and checked by the compiler as this document builds. Each structure below is followed immediately by the code that defines it; nothing is described here that is not in that code.

5.1 The augmented state

The state of a closed-loop agentic system, at a point in time, is modeled as a tuple of a latent computational component, a discrete operating mode, a memory/context/provenance component, a capability component, an environment component, and the state of a specification monitor (Chapter 4).

```
structure CTVGState (X Q M C E RPhi : Type) where
  latent      : X
  mode        : Q
  memory      : M
  capability  : C
  environment : E
  monitor     : RPhi
```

This is a product type and nothing more: the compiler checks that it forms a valid structure, which is a weaker guarantee than any theorem about it, and no theorem about `CTVGState` is proved anywhere in this thesis.

5.2 Belief as an interface

A belief is modeled here as an opaque summary type together with the two operations a monitor needs: reading off a point estimate, and updating on an observation.

```
structure Belief (S Y : Type) where
  estimate : S
  update   : Y → S
```

This is deliberately not a probability measure. A concrete instance (particle filter, Gaussian belief, finite-support posterior) would refine `estimate` and `update`; nothing in this thesis provides such an instance, and no claim here depends on which refinement a future instance would choose.

5.3 Intervention and composition

An agent proposes an action; a defender's intervention either lets it through or replaces it.

```
inductive Intervention (A : Type) where
  | allow
  | modify (a' : A)
  | sandbox (a' : A)
  | reducePrivilege (a' : A)
  | askHuman
  | deny
  | fallback

def compose {A : Type} (proposed : A) : Intervention A → A
  | .allow              => proposed
  | .modify a'          => a'
  | .sandbox a'         => a'
  | .reducePrivilege a' => a'
  | .askHuman           => proposed
  | .deny               => proposed
  | .fallback           => proposed
```

`compose` is total and type-checks; that is the entire content of this section. The three cases that currently fall through to `proposed` (`askHuman`, `deny`, `fallback`) do so because no distinguished "safe default" action has been fixed at this level of generality — fixing one would require committing to a concrete `Action` type, which this chapter does not do.

5.4 Capability ordering

Capability sets are given a partial order: `c1 ≤ c2` means every action `c1` allows, `c2` also allows.

```
structure CapabilitySet (Action : Type) where
  executable : Action → Bool

def CapabilitySet.le {Action : Type} (c1 c2 : CapabilitySet
↪  Action) : Prop :=
  ∀ a, c1.executable a = true → c2.executable a = true
```

```
instance {Action : Type} : LE (CapabilitySet Action) :=
↪  ⟨CapabilitySet.le⟩
```

This ordering is what Chapter 5's theorem is stated against. It is a `def` and an `instance`, checked for type-correctness by the compiler; the theorem that gives this ordering its content is proved separately, in Chapter 5, and is not restated here.

6 The Specification Automaton

Not every hazard is a property of a single state. Reading an attacker-controlled document, extracting a secret, and transmitting it to an attacker-controlled endpoint violates no single-state predicate; the violation is a property of the trajectory. This chapter gives the one Lean object this thesis uses to reduce that kind of trajectory-level question to an ordinary state-level one.

6.1 The automaton

A safety specification is compiled into a deterministic monitor with a start state, a step function reading the current state, the action taken, and the resulting state, and a predicate marking which of its states are rejecting.

```
structure SpecAutomaton (R S A : Type) where
  init    : R
  step    : R → S → A → S → R
  reject  : R → Bool

/-- Running the automaton over a whole trajectory, so that a
↪  trajectory
violates the specification exactly when the automaton ends in
↪  a rejecting
state. -/
def SpecAutomaton.run {R S A : Type} (spec : SpecAutomaton R S
↪  A)
    (states : List S) (actions : List A) : R :=
  match states, actions with
  | s :: s' :: states', a :: actions' =>
      spec.step (SpecAutomaton.run spec (s :: states')
      ↪  actions') s a s'
  | _, _ => spec.init
```

`SpecAutomaton.run` type-checks and its termination is accepted by the compiler's structural-recursion check on the argument lists; nothing beyond that is claimed. In particular, this chapter does not prove that any concrete instantiation of `step` correctly compiles any particular temporal formula, does

not prove `SpecAutomaton.run` correct against any semantics for trajectories, and does not prove that `reject` classifies trajectories the way an informal safety specification would want. What is proved is only that the automaton, as a piece of Lean code, is well-typed and terminates — which is exactly the reduction Chapter 3's `CTVGState.monitor` field presupposes is available: the monitor component of the state is intended to be populated by `SpecAutomaton.run`'s current partial result, so that a trajectory-level property becomes the ordinary state-level question of whether that field currently satisfies `reject`.

7 The Shield, and Capability Monotonicity

This chapter contains the one theorem in this thesis that is a genuine machine-checked proof rather than a definition, an interface, or a conjecture.

7.1 The shield interface

A shield checks an agent's proposed action against a hard admissibility predicate and a separate, second-stage predicate, and substitutes the best available admissible alternative when the proposal fails either check.

```
/-- A shield interface: `hardInvariant` and `valueBound` are
↪  left abstract,
so that this structure fixes only the control flow — check,
↪  then fall back
to the best admissible alternative — not any concrete safety
↪  criterion. -/
structure Shield (Action : Type) where
  hardInvariant : Action → Bool
  valueBound    : Action → Bool
  utility       : Action → Float
  candidates    : List Action

def Shield.admissible {Action : Type} (shield : Shield Action)
↪  : List Action :=
  shield.candidates.filter (fun a => shield.hardInvariant a &&
  ↪  shield.valueBound a)

def Shield.decide {Action : Type} (shield : Shield Action)
↪  (proposed : Action)
    (isProposedAdmissible : Bool) : Action :=
  if isProposedAdmissible then proposed
  else
    let admissible := shield.admissible
    match admissible with
```

```
    | []      => proposed  -- no fallback is fixed at this
    ↪  level of generality
    | a :: as =>
      as.foldl (fun best a =>
        if shield.utility a > shield.utility best then a else
        ↪  best) a
```

`Shield.decide`'s empty-admissible-set branch returns the unmodified proposal, which is very likely the wrong thing to do in a deployed system. No theorem in this thesis says what the right thing is, and no default is fixed here beyond what type-checks.

7.2 Capability monotonicity

Reachability in a finite number of steps, under a given capability set, is defined as an inductive predicate: a state reaches itself in zero steps, and reaches a further state by one more executable-action step.

```
inductive ReachableIn {State Action : Type} (step : State →
↪  Action → List State)
    (c : CapabilitySet Action) : State → State → Prop where
  | refl (s : State) : ReachableIn step c s s
  | tail {s0 s' s : State} (a : Action) :
      ReachableIn step c s0 s' →
      c.executable a = true →
      s ∈ step s' a →
      ReachableIn step c s0 s

theorem capability_reachable_subset {State Action : Type}
    (step : State → Action → List State)
    (c1 c2 : CapabilitySet Action) (h : c1.le c2)
    (s0 s : State) :
    ReachableIn step c1 s0 s → ReachableIn step c2 s0 s := by
  intro hr
  induction hr with
  | refl => exact ReachableIn.refl s0
  | tail a hprev hexec hstep ih =>
      exact ReachableIn.tail a ih (h a hexec) hstep
```

`capability_reachable_subset` is checked by the Lean kernel: if `c1 ≤ c 2` in the sense of Chapter 3's capability ordering, every state reachable under `c1` is reachable under `c2`. Its content is exactly this statement and no more. It does not say anything about probability, about a defender's ability to control what is reached, about safety, or about usefulness; it is a fact about monotone reachability graphs, stated here because the capability ordering it needs was already in scope, and its relevance to agentic AI safety is entirely in the reading

— shell access, unrestricted networking, credentials, write access, and delegation are each naturally modeled as enlarging the executable-action set at every state that has them — a reading this thesis asserts but does not, and could not, prove.

8 A Recoverability-Monitor Interface

An earlier draft of this thesis defined recoverability as a probability — the best-case-over-defender, worst-case-over-adversary chance of returning to a confidently safe core region before failing — and asserted, without proof, that crossing a resulting boundary was the event most worth monitoring in a deployed agent. That probability was never computed, no existence result was given for it, and no Lean object backed it, so under this thesis's rule it was removed (Chapter 7). What remains is the interface a monitor implementing some future, actual recoverability estimate would need.

```
/-- An interface for a recoverability monitor: given a current
↪  estimate and
an observation, produce the next estimate, and flag the event
↪  that the
estimate has dropped from at or above a threshold to below it.
↪  Nothing here
computes, bounds, or justifies `estimate`; the structure only
↪  fixes what a
caller would pass in and get back. -/
structure RecoverabilityMonitor (Y : Type) where
  estimate : Float
  update   : Y → Float

def crossedPointOfNoReturn (eta prevEstimate newEstimate :
↪  Float) : Bool :=
  (1.0 - eta ≤ prevEstimate) && (newEstimate < 1.0 - eta)
```

`RecoverabilityMonitor` and `crossedPointOfNoReturn` type-check; that is the whole of this chapter's content. `crossedPointOfNoReturn` is a pure comparison of two floating-point numbers against a threshold — it says nothing about what `estimate` should be computed from, and no instance of `RecoverabilityMonitor` is constructed anywhere in this thesis. The chapter is included, rather than cut along with the rest of the recoverability material, be-

cause it is genuinely Lean: two declarations that type-check against Chapter 3's structures and nothing more.

9 What Was Cut, and Why

This thesis was rewritten under a rule adopted from its companion volume: a claim survives only if it is proved in an elaborated Lean block or resolves to a named declaration. Applying that rule to the original proposal removed most of it. This chapter lists what was removed, so the cut is auditable rather than quiet.

9.1 The robust failure value and the viability kernel

Proposed: a value function `V_T(B)`, computed as the smallest, over defender policies, of the largest, over adversary policies and plausible transition models, probability of a trajectory reaching a failure set before a completion set; a "useful viability kernel" of belief states from which some defender policy keeps this value and a companion tail-severity functional below fixed tolerances while still finishing the task; and a "critical surface" at the kernel's boundary.

Cut, in full. No transition model, no defender-policy type, no adversary-policy type, and no value function were ever defined in Lean. The recursion proposed to compute `V_T` was written down by analogy with classical Bellman recursions and its existence and convergence were never established, even informally, let alone proved.

9.2 Tail severity and adversarial energy

Proposed: a CVaR-based tail-severity functional, and a "critical energy" measuring the minimum adversarial effort needed to force a belief state out of the viability kernel.

Cut, in full, for the same reason: both were defined only in prose, against objects (the viability kernel, an attacker effort function) that themselves were never formalized.

9.3 Recoverability as a probability, and the hysteresis prediction

Proposed: a recoverability functional `R_T(B)`, a "core" safe region, a resulting point-of-no-return boundary, and a prediction that memory-bearing agents would show measurable hysteresis in failure probability under a forward/backward sweep of a hazard-intensity parameter, while stateless agents would not.

Cut. What is kept, in Chapter 6, is only the bookkeeping interface a future monitor implementing some such quantity would expose — an estimate and an update function — with no claim about what the estimate should be, how it would be computed, or whether the hysteresis prediction is true. The prediction itself, being an empirical claim about systems that were never built or measured as part of this thesis, does not belong in a document gated on Lean backing, and is removed rather than retained as a labeled conjecture.

9.4 The runtime-assurance fallback rule

Proposed: a design rule that, when a shield's admissible-action set is empty, control should switch to a distinct, independently-certified fallback controller, by analogy with the Simplex architecture from safety-critical control.

Cut as a rule. What is kept, in Chapter 5, is the observation that `Shield.decide`'s empty-admissible-set branch currently returns the unmodified proposal — a fact about the Lean code, stated as a fact about the code, not as a design recommendation about what a real system should do instead.

9.5 Causal-incentive analysis and the Defense Trilemma reading

Proposed: a reading of structural causal influence diagrams as complementary to reachability analysis, identifying "pressure" where reachability identifies "consequence"; and a reading of the companion thesis's Defense Trilemma result as motivating this thesis's (now-removed) shield architecture.

Cut, in full. Neither reading was a claim this thesis could back with a declaration — the first cites no diagram formalized anywhere in this project, and the second is commentary on a different thesis's theorem, not a claim proved here.

9.6 The eight-item empirical identification program

Proposed: a procedure for discovering an adequate state representation by an empirical Markov-sufficiency test, a procedure for fitting a transition model and an ambiguity set from data, a procedure for estimating `V_T` and testing

for a critical surface, and eight falsifiable predictions collected from the rest of the (now-removed) theory.

Cut, in full. No data was collected, no model was fit, and every prediction depended on a quantity — `V_T`, `E_crit`, `R_T`, the hysteresis loop area — that no longer appears in this thesis.

9.7 What this leaves

What survives is Chapters 3 through 6: four Lean structures, one inductive predicate, and one theorem. That is the whole of this thesis's formal content, and Chapter 8 says so without padding it back out.

10 Conclusion

This thesis, in its current form, formalizes a small piece of vocabulary for reasoning about agentic AI safety — an augmented state, a belief interface, an intervention/composition mechanism, a capability partial order, a specification automaton, a shield interface, and a recoverability-monitor interface — and proves one theorem against that vocabulary: `capability_reachable_subset`, that a capability ordering which only ever adds executable actions can only ever enlarge what is reachable.

That is a modest result relative to the theory originally proposed. It is stated as such rather than dressed up, because the alternative — keeping the larger theory's definitions and conjectures in the document, labeled as conjectures — was tried in an earlier draft and rejected: a claim that cannot be checked by the Lean kernel does not belong in a thesis gated on Lean backing, no matter how carefully it is hedged in prose. Chapter 7 records exactly what that rejection removed, so that the removal is inspectable rather than a matter of trust.

What this thesis takes itself to have shown is narrower than what its companion volume shows for hallucination and prompt-injection defense: a type-correct formal skeleton, and one theorem within it. The larger claims about robust reachability values, viability kernels, critical surfaces, recoverability, adversarial energy, and hysteresis in memory-bearing agents remain exactly what they were before this rewrite — informal proposals, possibly worth formalizing — and this document no longer asserts them.